

MATLAB Simulation Codes for [1, Sect. VI]

These MATLAB codes reproduce the simulation results from [1, Sect. VI]. In addition to the scripts `ExA.m` and `ExB.m`, the codes contain auxiliary functions `powerseries2x2.m`, `powerseries4D.m`, `controlgain.m`, and `nmFD.m`. The simulation results from [1, Sect. VI] can be reproduced by running the script files which employ the auxiliary function files.

Description

Large-scale systems of $n + m$ hyperbolic PDEs can be written as

$$\mathbf{u}_t(t, x) + \mathbf{\Lambda}(x)\mathbf{u}_x(t, x) = \frac{1}{n}\mathbf{\Sigma}(x)\mathbf{u}(t, x) + \frac{1}{m}\mathbf{W}(x)\mathbf{v}(t, x), \quad (1a)$$

$$\mathbf{v}_t(t, x) - \mathbf{M}(x)\mathbf{v}_x(t, x) = \frac{1}{n}\mathbf{\Theta}(x)\mathbf{u}(t, x) + \frac{1}{m}\mathbf{\Psi}(x)\mathbf{v}(t, x), \quad (1b)$$

with boundary conditions

$$\mathbf{u}(t, 0) = \frac{1}{m}\mathbf{Q}\mathbf{v}(t, 0), \quad \mathbf{v}(t, 1) = \frac{1}{n}\mathbf{R}\mathbf{u}(t, 1) + \mathbf{U}(t), \quad (2)$$

where $\mathbf{u} = (u_i)_{i=1}^n$, $\mathbf{v} = (v_j)_{j=1}^m$ are the states, and $\mathbf{U} = (U_j)_{j=1}^m$ is the control input. The respective class of $\infty + \infty$ continuum PDEs is of the form

$$u_t(t, x, y) + \lambda(x, y)u_x(t, x, y) = \int_0^1 \sigma(x, y, \zeta)u(t, x, \zeta)d\zeta + \int_0^1 W(x, y, \zeta)v(t, x, \zeta)d\zeta, \quad (3a)$$

$$v_t(t, x, \eta) - \mu(x, \eta)v_x(t, x, \eta) = \int_0^1 \theta(x, \eta, \zeta)u(t, x, \zeta)d\zeta + \int_0^1 \psi(x, \eta, \zeta)v(t, x, \zeta)d\zeta, \quad (3b)$$

with boundary conditions

$$u(t, 0, y) = \int_0^1 Q(y, \zeta)v(t, 0, \zeta)d\zeta, \quad v(t, 1, \eta) = \int_0^1 R(\eta, \zeta)u(t, 1, \zeta)d\zeta + U(t, \eta), \quad (4)$$

which acts as continuum approximation of the large-scale system.

The continuum system (3), (4) can be exponentially stabilized by a backstepping state feedback law of the form

$$U(t, \eta) = - \int_0^1 R(\eta, \zeta)u(t, 1, \zeta)d\zeta + \int_0^1 \int_0^1 K(1, \xi, \eta, \zeta)u(t, \xi, \zeta)d\zeta d\xi + \int_0^1 \int_0^1 L(1, \xi, \eta, \zeta)v(t, \xi, \zeta)d\zeta d\xi, \quad (5)$$

where K, L satisfy the kernel equations

$$\begin{aligned} \mu(x, \eta)K_x(x, \xi, \eta, \zeta) - K_\xi(x, \xi, \eta, \zeta)\lambda(\xi, \zeta) - K(x, \xi, \eta, \zeta)\lambda_\xi(\xi, \zeta) = \\ \int_0^1 K(x, \xi, \eta, \chi)\sigma(\xi, \chi, \zeta)d\chi + \int_0^1 L(x, \xi, \eta, \chi)\theta(\xi, \chi, \zeta)d\chi, \end{aligned} \quad (6a)$$

$$\begin{aligned} \mu(x, \eta)L_x(x, \xi, \eta, \zeta) + L_\xi(x, \xi, \eta, \zeta)\mu(\xi, \zeta) + L(x, \xi, \eta, \zeta)\mu_\xi(\xi, \zeta) = \\ \int_0^1 K(x, \xi, \eta, \chi)W(\xi, \chi, \zeta)d\chi + \int_0^1 L(x, \xi, \eta, \chi)\psi(\xi, \chi, \zeta)d\chi, \end{aligned} \quad (6b)$$

with boundary conditions

$$L(x, x, \eta, \zeta) = \frac{\psi(x, \eta, \zeta)}{\mu(x, \zeta) - \mu(x, \eta)}, \quad (7a)$$

$$K(x, x, \eta, \zeta) = -\frac{\theta(x, \eta, \zeta)}{\lambda(x, \zeta) + \mu(x, \eta)}, \quad (7b)$$

$$L(x, 0, \eta, \zeta) = \frac{1}{\mu(0, \zeta)} \int_0^1 K(x, 0, \eta, \chi) \lambda(0, \chi) Q(\chi, \zeta) d\chi, \quad \forall \eta \leq \zeta, \quad (7c)$$

$$L(1, \xi, \eta, \zeta) = l(\xi, \eta, \zeta), \quad \zeta < \eta, \quad (7d)$$

where (7d) is an artificial boundary condition chosen as

$$l(\xi, \eta, \zeta) = \frac{\psi(\xi, \eta, \zeta)}{\mu(\xi, \zeta) - \mu(\xi, \eta)}, \quad \forall \zeta < \eta. \quad (8)$$

The paper [1] introduces two micro-macro controllers for the $n + m$ system (1), (2), which are illustrated by the numerical examples. The first designed control law employs micro controls with macro kernels and micro measurements, and it is of the form

$$\mathbf{U}(t) = -\frac{1}{n} \mathbf{R} \mathbf{u}(t, 1) + \frac{1}{n} \int_0^1 \tilde{\mathbf{K}}(1, \xi) \mathbf{u}(t, \xi) d\xi - \frac{1}{m} \int_0^1 \tilde{\mathbf{L}}(1, \xi) \mathbf{v}(t, \xi) d\xi, \quad (9)$$

where $\tilde{\mathbf{K}} = (\tilde{K}_{i,j})_{i=1}^m_{j=1}^n$, $\tilde{\mathbf{L}} = (\tilde{L}_{i,j})_{i,j=1}^m$ are obtained through sampling of the continuum kernels as¹

$$\tilde{K}_{i,j}(1, \xi) = K(1, \xi, i/n, j/m), \quad (11a)$$

$$\tilde{L}_{i,j}(1, \xi) = L(1, \xi, i/n, j/m). \quad (11b)$$

The second designed control law employs micro controls with macro kernels and macro measurements, where approximate (macro) state information $(\tilde{u}, \tilde{v})$ is assumed to be available instead of the exact (micro) state information $(\mathbf{u}, \mathbf{v})$. The respective control law is of the form

$$\mathbf{U}(t) = -\int_0^1 \tilde{\mathbf{R}}(\zeta) \tilde{u}(t, 1, \zeta) d\zeta + \int_0^1 \int_0^1 \mathbf{K}(1, \xi, \zeta) \tilde{u}(t, \xi, \zeta) d\xi d\zeta + \int_0^1 \int_0^1 \mathbf{L}(1, \xi, \zeta) \tilde{v}(t, \xi, \zeta) d\xi d\zeta, \quad (12)$$

¹In theory, the mean-value sampling

$$\tilde{K}_{i,j}(1, \xi) = nm \int_{(i-1)/m}^{i/m} \int_{(j-1)/n}^{j/n} K(1, \xi, \eta, \zeta) d\zeta d\eta, \quad (10a)$$

$$\tilde{L}_{i,j}(1, \xi) = m^2 \int_{(i-1)/m}^{i/m} \int_{(j-1)/m}^{j/m} L(1, \xi, \eta, \zeta) d\zeta d\eta, \quad (10b)$$

would be a more appropriate sampling method, but as the numerical examples deal with (piecewise) continuous kernels, the pointwise sampling (11) is employed in the code.

with $\widetilde{\mathbf{R}} = (\widetilde{R}_i)_{i=1}^m$ given by (in the numerical examples, $\mathbf{R} = 0$ so that this step is omitted in the code)

$$\widetilde{R}_i(\zeta) = m \int_{(i-1)/m}^{i/m} R(\chi, \zeta) d\chi, \quad (13)$$

and $\mathbf{K} = (K_i)_{i=1}^m$, $\mathbf{L} = (L_i)_{i=1}^m$ given by²

$$K_i(1, \xi, \zeta) = K(1, \xi, i/m, \zeta), \quad (15a)$$

$$L_i(1, \xi, \zeta) = L(1, \xi, i/m, \zeta). \quad (15b)$$

The functions

```
[Ks, Ls, rd] = powerseries2x2(Lf, Mf, Sf, Wf, THf, Q, N)
[K0s, K1s, K2s, L0s, L1s, L2s, rd] = powerseries4D(Lf, Mf, Sf, Wf, THf, PSf, Qf, N, msg)
```

solve scalar 2×2 and 4-D continuum kernel equations, respectively based on the power series method [2, 3]. The implementations are adapted from a respective implementation for solving $\infty + 1$ kernel equations from github.com/jphumaloja/Continuum-Kernels-Power-Series. The method approximates the solution to the kernel equations (6), (7) as power series (respectively for L)

$$K(x, \xi, \eta, \zeta) = \sum_{i=0}^N \sum_{j=0}^{N-i} \sum_{k=0}^{N-i-j} \sum_{\ell=0}^k c_{i,j,k,\ell} x^\ell \xi^{k-\ell} \eta^j \zeta^i, \quad (16)$$

(η, ζ are absent from 2×2 kernel equations) for given parameters. In case of 4-D kernel equations, the power series approximations are split into three parts corresponding to the subdomains of the kernel equations (see [1, (75)–(77)] for details). The final output `rd` returns the residual of the power series approximation.

The function

```
[A, B] = nmFD(lamy, muy, sigy, Wy, thy, psiy, qy, n, m, xg)
```

constructs a finite-difference approximation $\dot{\mathbf{X}}(t) = \mathbf{A}\mathbf{X}(t) + \mathbf{B}\mathbf{U}(t)$ for an $n + m$ system (1), (2) based on given (continuum) parameters using `xg` grid points for spatial discretization. The discretizations for the $\mathbf{u}$ - and $\mathbf{v}$ -channels employ forward- and backward-differences, respectively, so that the computational direction is consistent with the transport direction of the PDE.

The function

²In theory, the mean-value sampling

$$K_i(1, \xi, \zeta) = m \int_{(i-1)/m}^{i/m} K(1, \xi, \chi, \zeta) d\chi, \quad (14a)$$

$$L_i(1, \xi, \zeta) = m \int_{(i-1)/m}^{i/m} L(1, \xi, \chi, \zeta) d\chi, \quad (14b)$$

would be a more appropriate sampling method, but as the numerical examples deal with (piecewise) continuous kernels, the pointwise sampling (15) is employed in the code.

`[Km, Lm] = controlgain(muyn, m, xg, K0s, K1s, K2s, L0s, L1s, L2s)`

constructs the backstepping control gains $\tilde{\mathbf{K}}, \tilde{\mathbf{L}}$ from (9) corresponding to a finite-difference approximation of an $n + m$ system constructed with `nmFD`, and based on 4-D kernels K, L computed with `powerseries4D.m`

Examples

The simulation results for [1, Sect. VI.A] and [1, Sect. VI.B] can be reproduced by running the scripts `ExA.m` and `ExB.m`, respectively.

References

- [1] J.-P. Humaloja and N. Bekiaris-Liberis. Micro-macro backstepping control of large-scale hyperbolic systems. *arXiv*, 2510.12456, 2025.
- [2] J.-P. Humaloja and N. Bekiaris-Liberis. On computation of approximate solutions to large-scale backstepping kernel equations via continuum approximation. *Syst. Control Lett.*, 196:105982, 2025.
- [3] R. Vazquez, G. Chen, J. Qiao, and M. Krstic. The power series method to compute backstepping kernel gains: theory and practice. In *IEEE Conf. Decis. Control*, pages 8162–8169, 2023.